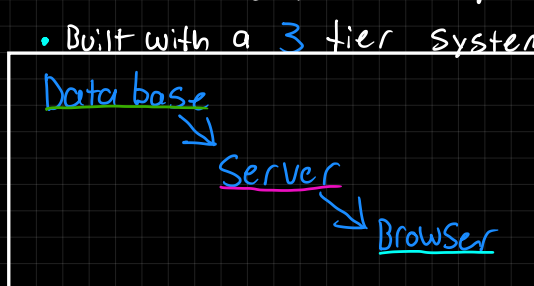
XenFlo: What is it?

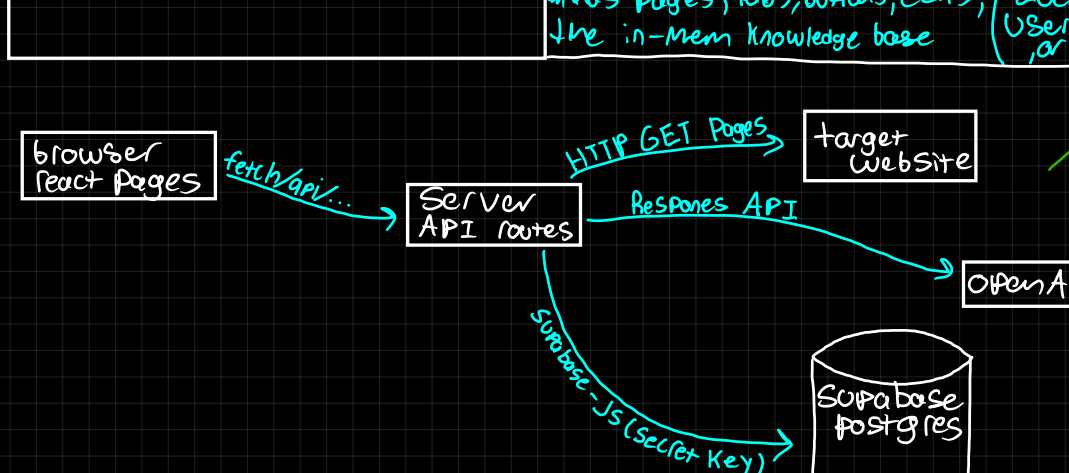
More documentation available in repo
in same directory as these notes

- In short it's a URL scraper (a program that reads data from sites)
- Built with a 3 tier system



What does it have?

- The Database has saved knowledge bases, version history, AI cache, daily AI counter (Supabase ∴ a hosted Postgres service) (Hosted → someone elses; Postgres → a database system program)
- The Server has the scraping logic, AI calls, db writes: anything that needs secrets/is slow → (Vercel server or a local server → npm run dev)
- has pages, tabs, buttons, edits, the in-mem knowledge base (Locally on users laptop, pc, or phone)



Why is it built like this?

the browser never talks to Supabase (db), or OpenAI directly. Both have secret keys if they were in the browser than anyone could read it!

Server Info

Why vercel vs githubpages? ∴ Because XenFlow requires a server - to run the scrape externally ∴ of CORS/SOP since servers dont care about CORS/SOP... → we could've still done gh pages with Cloud flare workers, but thats double the work so why?

- Origin.. 3 parts
  1. https:// (the protocol)
  2. xenflo-phi.vercel.app (the domain)
  3. Port # (hidden)

  all have to match to be same origin
- permission ∴ two sites A/B, A wants data from B. B can say sure (Access-Control-Allow-Origin: https://site-a.com) ∴ Browser then lets A read the data. the permission line is CORS/allow (what it chooses to share)
- API'S LIVE IN A DOMAIN!!!

SOP +

What is CORS ∴ Cross Origin Resource Sharing + Same Origin Policy

- CORS ∴ allows other sits to read specific info on a site.
- SOP ∴ The default that says no

SQL + Postgres

SQL ∴ is the Language/Logic for the Commands

Select name, city from knowledge_bases where city = 'Las Vegas'

Requests city names of knowledge bases BUT IT DOESN'T KNOW HOW

Postgres ∴ Contains the knowledge of how to complete the task
↳ BUT I CAN!

Why Seperate? ∴ The "Best" (hows) methods keep changing

01 · tech stack + concepts

Next.js ∴ framework on top of react
- urls (routing)
- a server
- a compiler for browser output → vercel

App Router ∴ in app/, recieves + directs requests and hands it to the correct file route.ts (serve code that answers HTTP requests no UI) → how, what type, where, how much of...

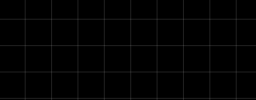
React 19 ∴ UI out of components (a custom tag you define) functions that return a markup (JSX) and rerun when their state changes

TypeScript ∴ (like struct objects in C++) JS + types. Strict → like a compiler setting, that requires better, more explicit code.

Tailwind CSS v4 ∴ Styles elements with small class names in the markup

Framer Motion ∴ animation lib for react

Cheerio ∴ Loads an HTML string and lets you query it with Jquery → selects elements and copies them like



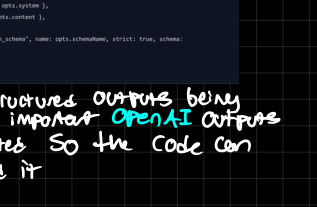
```
footer { color: gray; }     /* every <footer> */
.btn { padding: 8px; }      /* every element with class "btn" */
#contact { margin: 0; }     /* the element with id "contact" */
```

1. `$("footer").text()`
   a. `$("footer")` finds the `<footer>` element.
   b. `.text()` gets all the plain text inside it, like "© 2026 by Dragon Factory. All Right Reserved."
   c. That's how the business facts extractor reads copyright lines.
2. `$('a[href^="tel:"]')`
   a. `a` means links (`<a>` tags).
   b. `[href^="tel:"]` is an attribute selector: only links whose `href` starts with (`^=`) `"tel:"`.
   c. So it finds phone links like `<a href="tel:7022919893">Call us</a>`.
   d. That's how the phone extractor finds clickable phone numbers.

Zod ∴ keeps the struct like definition with its datatypes while the program runs typeScript doesn't do this

Supabase ∴ SQL db + Postgres requires to save data

OpenAI Response API ∴



```
// src/lib/ai/openai.ts:36-52
const res = await fetch("https://api.openai.com/v1/responses", {
  method: "POST",
  signal: controller.signal,
  headers: { authorization: `Bearer ${key}`, "content-type": "application/json" },
  body: JSON.stringify({
    model: opts.model,
    // No hidden reasoning: reasoning tokens count toward max_output_tokens, and these are
    // summarize-and-format tasks where the whole budget should go to the answer.
    reasoning: { effort: "none" },
    max_output_tokens: opts.maxOutputTokens ?? LIMITS.outputTokens,
    store: false,
    input: [
      { role: "system", content: opts.system },
      { role: "user", content: opts.content },
    ],
    text: { format: { type: "json_schema", name: opts.schemaName, strict: true, schema:
toOpenAISchema(opts.schema) } },
  }),
});
```

With structured outputs being extremely important OpenAI outputs are formatted so the code can understand it

unpdf ∴ → only runs when requested. Serverless tool that converts pdf's → string of data

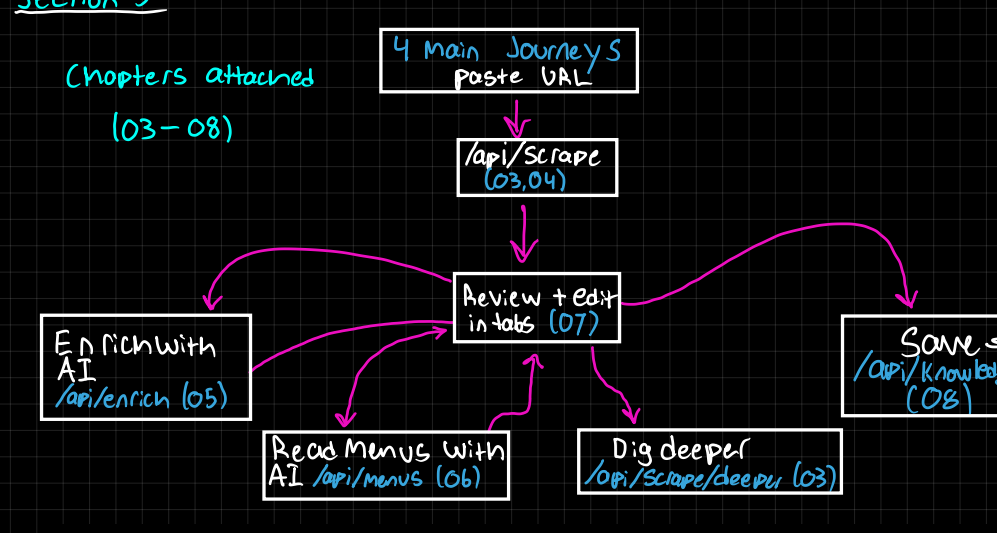
Vitest ∴ test runner 3rd party package that test outputs (normal examples, Edge cases, negative tests)

Server Comp vs Client Comp

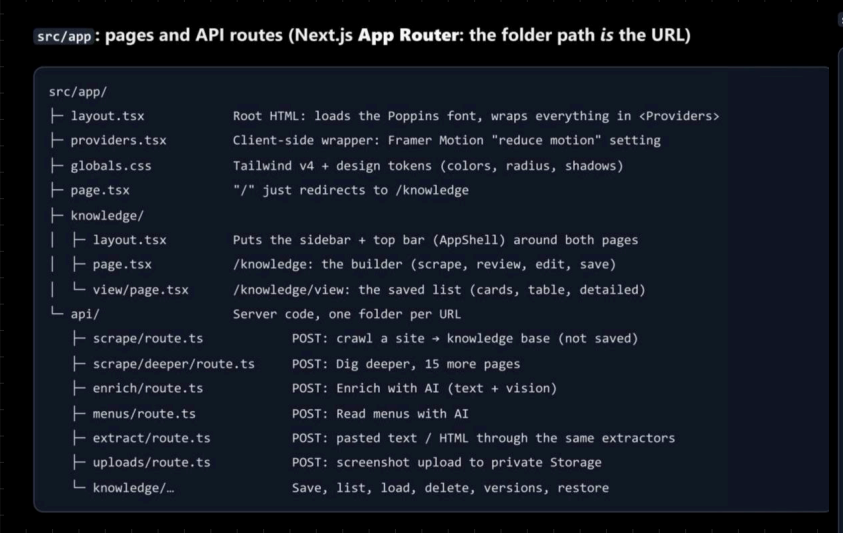
- in app Router components run on the server by default and output useable HTML to the browser NO CODE SENT
- Client ships to the browser CODE CAN BE SEEN

Section 3

Chapters attached (03–08)

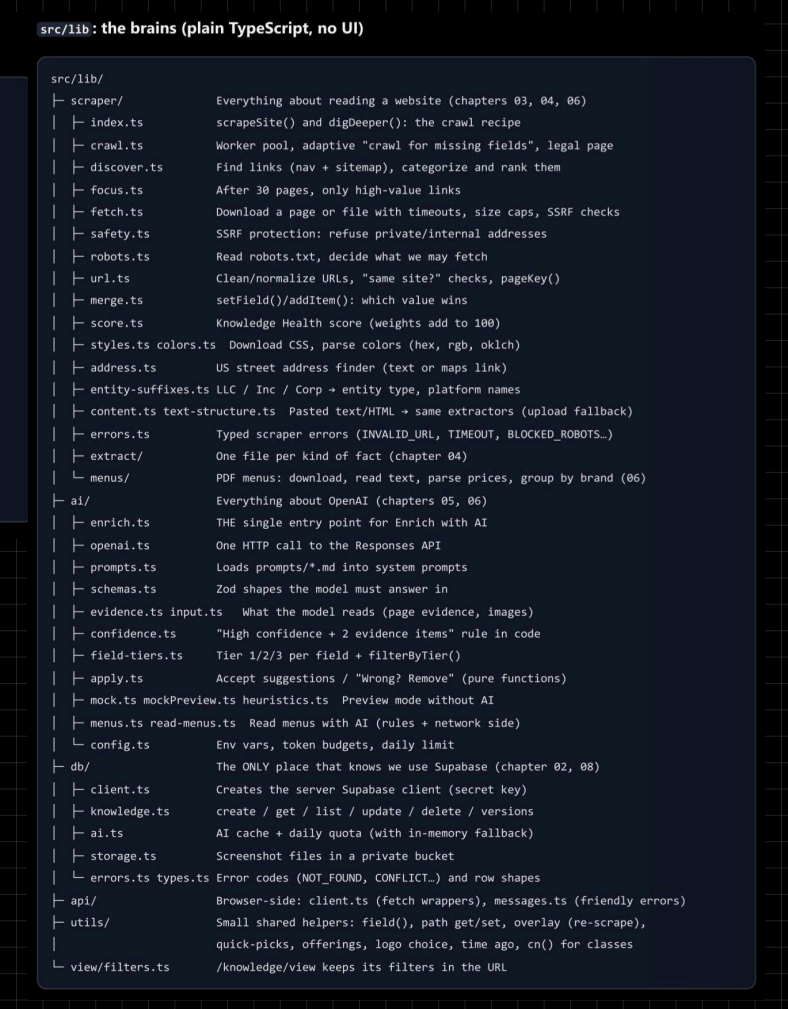


4 main journeys
paste URL
/api/scrape (03,04)
Review + edit in tabs (07)
Enrich with AI /api/enrich (05)
Read Menus with AI /api/menus (06)
Dig deeper /api/scrape/deeper (03)
Save → /api/knowledge (08)



`src/app`: pages and API routes (Next.js **App Router**: the folder path *is* the URL)

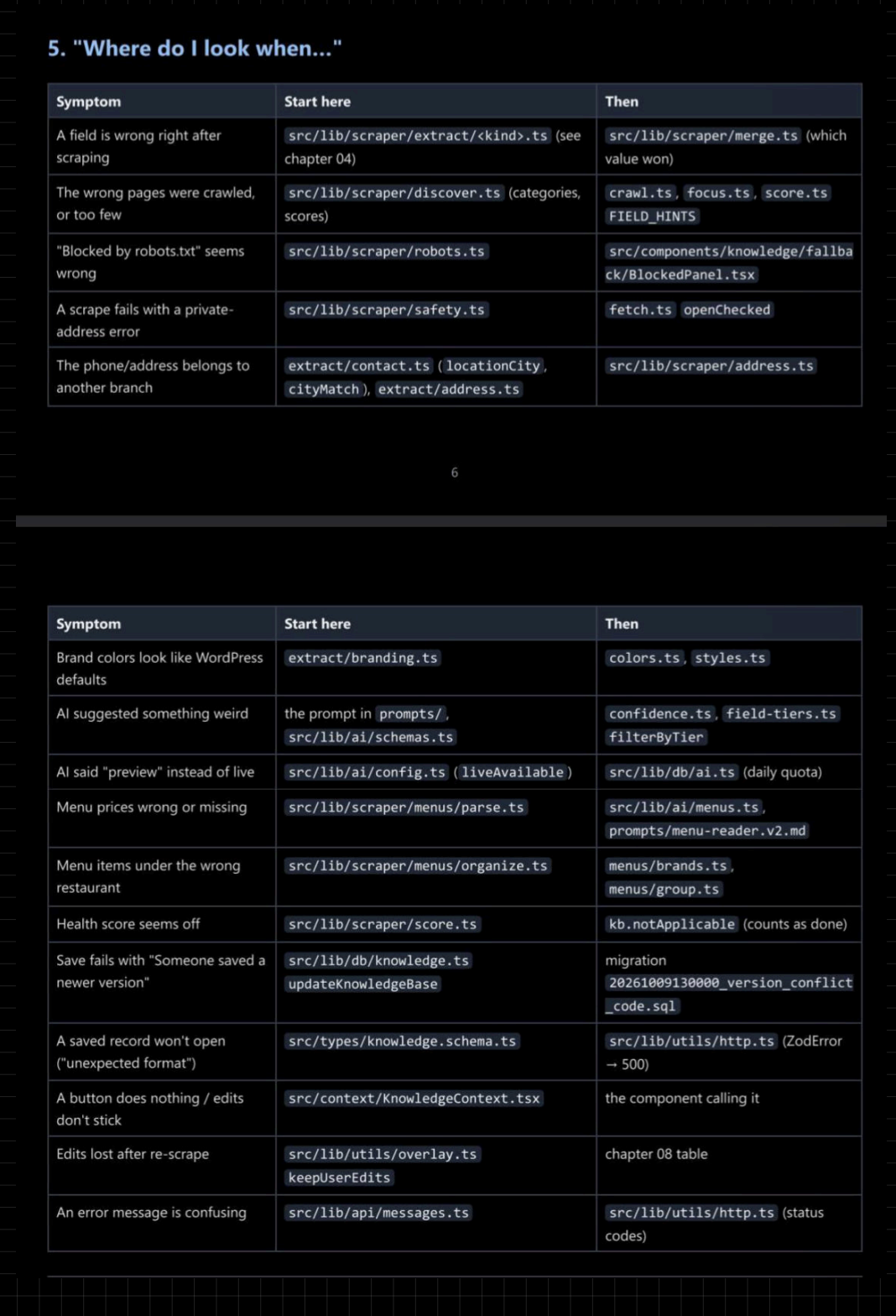
```
src/app/
├ layout.tsx          Root HTML: loads the Poppins font, wraps everything in <Providers>
├ providers.tsx       Client-side wrapper: Framer Motion "reduce motion" setting
├ globals.css         Tailwind v4 + design tokens (colors, radius, shadows)
├ page.tsx            "/" just redirects to /knowledge
├ knowledge/
│ ├ layout.tsx        Puts the sidebar + top bar (AppShell) around both pages
│ ├ page.tsx          /knowledge: the builder (scrape, review, edit, save)
│ └ view/page.tsx     /knowledge/view: the saved list (cards, table, detailed)
└ api/                Server code, one folder per URL
  ├ scrape/route.ts          POST: crawl a site → knowledge base (not saved)
  ├ scrape/deeper/route.ts   POST: Dig deeper, 15 more pages
  ├ enrich/route.ts          POST: Enrich with AI (text + vision)
```



`src/lib`: the brains (plain TypeScript, no UI)

```
src/lib/
├ scraper/            Everything about reading a website (chapters 03, 04, 06)
│ ├ index.ts          scrapeSite() and digDeeper(): the crawl recipe
│ ├ crawl.ts          Worker pool, adaptive "crawl for missing fields", legal page
│ ├ discover.ts       Find links (nav + sitemap), categorize and rank them
│ ├ focus.ts          After 30 pages, only high-value links
│ ├ fetch.ts          Download a page or file with timeouts, size caps, SSRF checks
│ ├ safety.ts         SSRF protection: refuse private/internal addresses
│ ├ robots.ts         Read robots.txt, decide what we may fetch
│ ├ url.ts            Clean/normalize URLs, "same site?" checks, pageKey()
│ ├ merge.ts          setField()/addItem(): which value wins
│ ├ score.ts          Knowledge Health score (weights add to 100)
│ ├ styles.ts colors.ts  Download CSS, parse colors (hex, rgb, oklch)
│ ├ address.ts        US street address finder (text or maps link)
│ ├ entity-suffixes.ts LLC / Inc / Corp → entity type, platform names
…
│ ├ evidence.ts input.ts  What the model reads (page evidence, images)
│ ├ confidence.ts     "High confidence + 2 evidence items" rule in code
│ ├ field-tiers.ts    Tier 1/2/3 per field + filterByTier()
│ ├ apply.ts          Accept suggestions / "Wrong? Remove" (pure functions)
│ ├ mock.ts mockPreview.ts heuristics.ts  Preview mode without AI
│ ├ menus.ts read-menus.ts  Read menus with AI (rules + network side)
│ └ config.ts         Env vars, token budgets, daily limit
├ db/                 The ONLY place that knows we use Supabase (chapter 02, 08)
│ ├ client.ts         Creates the server Supabase client (secret key)
│ ├ knowledge.ts      create / get / list / update / delete / versions
│ ├ ai.ts             AI cache + daily quota (with in-memory fallback)
│ ├ storage.ts        Screenshot files in a private bucket
│ └ errors.ts types.ts Error codes (NOT_FOUND, CONFLICT…) and row shapes
├ api/                Browser-side: client.ts (fetch wrappers), messages.ts (friendly errors)
├ utils/              Small shared helpers: field(), path get/set, overlay (re-scrape),
│                     quick-picks, offerings, logo choice, time ago, cn() for classes
└ view/filters.ts     /knowledge/view keeps its filters in the URL
```

Where do I look when...

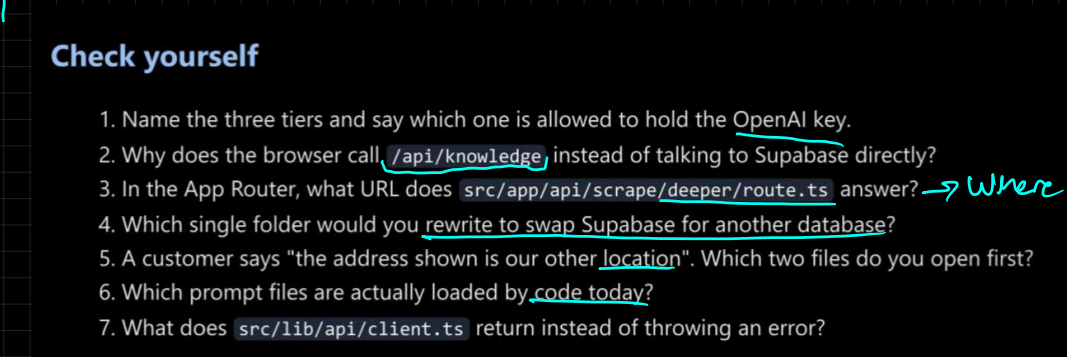


5. "Where do I look when..."

Symptom	Start here	Then
A field is wrong right after scraping	`src/lib/scraper/extract/<kind>.ts` (see chapter 04)	`src/lib/scraper/merge.ts` (which value won)
The wrong pages were crawled, or too few	`src/lib/scraper/discover.ts` (categories, scores)	`crawl.ts`, `focus.ts`, `score.ts` `FIELD_HINTS`
"Blocked by robots.txt" seems wrong	`src/lib/scraper/robots.ts`	`src/components/knowledge/fallback/BlockedPanel.tsx`
A scrape fails with a private-address error	`src/lib/scraper/safety.ts`	`fetch.ts` `openChecked`
The phone/address belongs to another branch	`extract/contact.ts` (`locationCity`, `cityMatch`), `extract/address.ts`	`src/lib/scraper/address.ts`

6

Symptom	Start here	Then
Brand colors look like WordPress defaults	`extract/branding.ts`	`colors.ts`, `styles.ts`
AI suggested something weird	the prompt in `prompts/`, `src/lib/ai/schemas.ts`	`confidence.ts`, `field-tiers.ts` `filterByTier`
AI said "preview" instead of live	`src/lib/ai/config.ts` (`liveAvailable`)	`src/lib/db/ai.ts` (daily quota)
Menu prices wrong or missing	`src/lib/scraper/menus/parse.ts`	`src/lib/ai/menus.ts`, `prompts/menu-reader.v2.md`
Menu items under the wrong restaurant	`src/lib/scraper/menus/organize.ts`	`menus/brands.ts`, `menus/group.ts`
Health score seems off	`src/lib/scraper/score.ts`	`kb.notApplicable` (counts as done)
Save fails with "Someone saved a newer version"	`src/lib/db/knowledge.ts` `updateKnowledgeBase`	migration `20261009130000_version_conflict_code.sql`
A saved record won't open ("unexpected format")	`src/types/knowledge.schema.ts`	`src/lib/utils/http.ts` (ZodError → 500)
A button does nothing / edits don't stick	`src/context/KnowledgeContext.tsx`	the component calling it
Edits lost after re-scrape	`src/lib/utils/overlay.ts`	[illegible]
An error message is confusing	`src/lib/api/messages.ts`	[illegible]

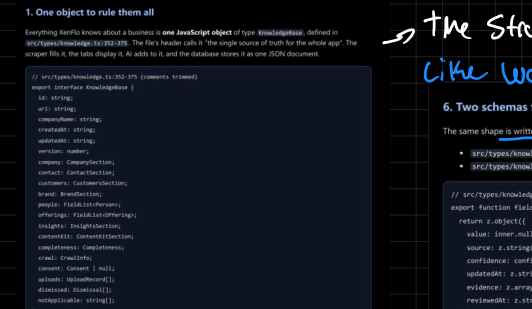


Check yourself

1. Name the three tiers and say which one is allowed to hold the OpenAI key.
2. Why does the browser call `/api/knowledge` instead of talking to Supabase directly?
3. In the App Router, what URL does `src/app/api/scrape/deeper/route.ts` answer? → Where does it write to...
4. Which single folder would you rewrite to swap Supabase for another database?
5. A customer says "the address shown is our other location". Which two files do you open first?
6. Which prompt files are actually loaded by code today?
7. What does `src/lib/api/client.ts` return instead of throwing an error?

1) Browser, Server, Database
2) Editing data on Supabase requires a Secret key
3) Code in deeper/route.ts gets called by browser and sends that data back to the browser that called it. that data goes through multiple processes and ends at KB
4) → src/lib/db/
5) extract/address.ts, extract/contact.ts
6) `understand-business.v1`, `logo-vision.v1`, `menu-reader.v2`, and `menu-reader.v1` (only for its cached answers). See `src/lib/ai/prompts.ts:11-16`. → Changed/updated prompts
7) → {data} or {error: {code, message}}.

02 the data model one object, two schemas, five tables



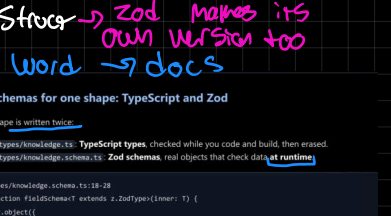
1. One object to rule them all

Everything XenFlo knows about a business is **one JavaScript object** of type `KnowledgeBase`, defined in `src/types/knowledge.ts:352-375`. The file's header calls it "the single source of truth for the whole app". The scraper fills it, the tabs display it, AI adds to it, and the database stores it as one JSON document.

```
// src/types/knowledge.ts:352-375 (comments trimmed)
export interface KnowledgeBase {
  id: string;
  url: string;
  companyName: string;
  createdAt: string;
  updatedAt: string;
  version: number;
  company: CompanySection;
  contact: ContactSection;
  customers: CustomersSection;
  brand: BrandSection;
  people: FieldList<Person>;
  offerings: FieldList<Offering>;
  insights: InsightsSection;
  contentKit: ContentKitSection;
  completeness: Completeness;
  crawl: CrawlInfo;
  consent: Consent | null;
  uploads: UploadRecord[];
  dismissed: Dismissal[];
  notApplicable: string[];
}
```

→ the struct → Zod makes its own version too

Like Word → docs



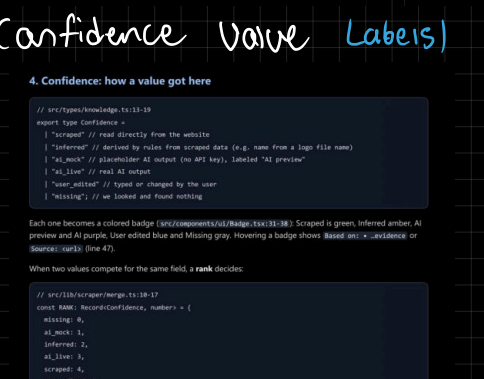
6. Two schemas for one shape: TypeScript and Zod

The same shape is written twice:

- `src/types/knowledge.ts`: **TypeScript types**, checked while you code and build, then erased.
- `src/types/knowledge.schema.ts`: **Zod schemas**, real objects that check data at runtime.

```
// src/types/knowledge.schema.ts:18-28
export function fieldSchema<T extends z.ZodType>(inner: T) {
  return z.object({
    value: inner.nullable(),
    source: z.string().nullable(),
    confidence: confidenceSchema,
    updatedAt: z.string(),
    evidence: z.array(z.string()).optional(),
    reviewedAt: z.string().optional(),
  });
}
```

Confidence Value Labels)



4. Confidence: how a value got here

```
// src/types/knowledge.ts:13-19
export type Confidence =
  | "scraped"    // read directly from the website
  | "inferred"   // derived by rules from scraped data (e.g. name from a logo file name)
  | "ai_mock"    // placeholder AI output (no API key), labeled "AI preview"
  | "ai_live"    // real AI output
  | "user_edited" // typed or changed by the user
  | "missing";   // we looked and found nothing
```

Each one becomes a colored badge (`src/components/ui/Badge.tsx:31-38`): Scraped is green, Inferred amber, AI preview and AI purple, User edited blue and Missing gray. Hovering a badge shows `Based on: • …evidence` or `Source: <url>` (line 47).

When two values compete for the same field, a **rank** decides:

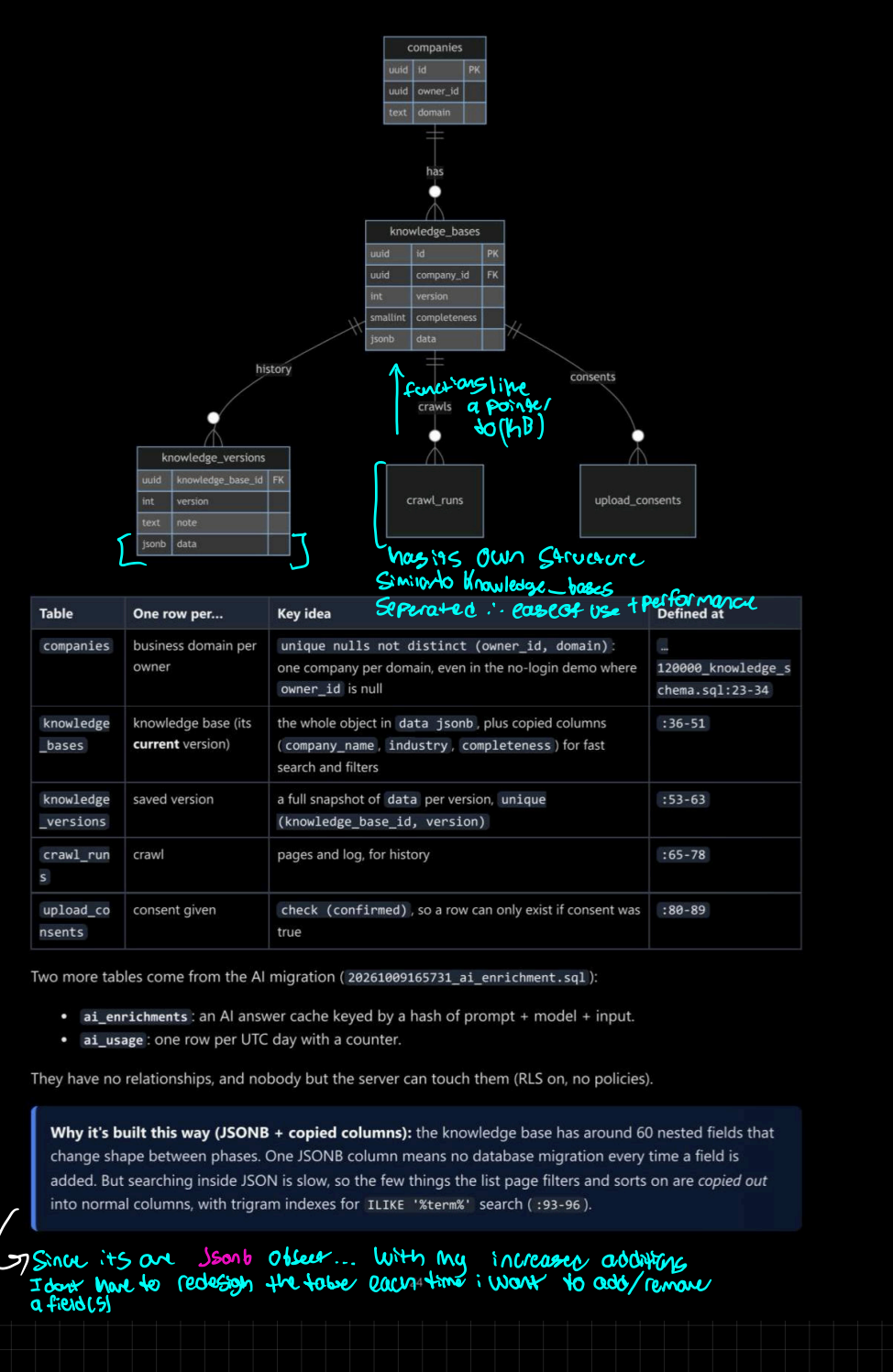
```
// src/lib/scraper/merge.ts:10-17
const RANK: Record<Confidence, number> = {
  missing: 0,
  ai_mock: 1,
  inferred: 2,
  ai_live: 3,
  scraped: 4,
  user_edited: 5,
};
```

prefe data thats scraped, used for labelling

5 What AI is allowed to fill:

	Meaning	Examples	AI May ...
Read 1.)	facts scraper extracts	Names, addresses, phones, logos, prices, FAQs etc...	Not overwrite, fill an empty if value appears word forword in evidence
Inferred 2)	Obvious to us, but not written into label	industry, business model, channels, pitch, writing style, content kit	fill, with evidence @ high confidence
Never guessed 3)	Never guessed priv facts/people	People, legal name, entity type, employee ct., revenue, testimonial authors	N/A

Database ∴ five tables + 2 helpers



functions like a pointer to (KB)

has its own structure similar to knowledge_bases

Seperated ∴ ease of use + performance

Table	One row per...	Key idea	Defined at
`companies`	business domain per owner	`unique nulls not distinct (owner_id, domain)`: one company per domain, even in the no-login demo where `owner_id` is null	`…120000_knowledge_schema.sql:23-34`
`knowledge_bases`	knowledge base (its **current** version)	the whole object in `data jsonb`, plus copied columns (`company_name`, `industry`, `completeness`) for fast search and filters	`:36-51`
`knowledge_versions`	saved version	a full snapshot of `data` per version, `unique (knowledge_base_id, version)`	`:53-63`
`crawl_runs`	crawl	pages and log, for history	`:65-78`
`upload_consents`	consent given	`check (confirmed)`, so a row can only exist if consent was true	`:80-89`

Two more tables come from the AI migration (`20261009165731_ai_enrichment.sql`):

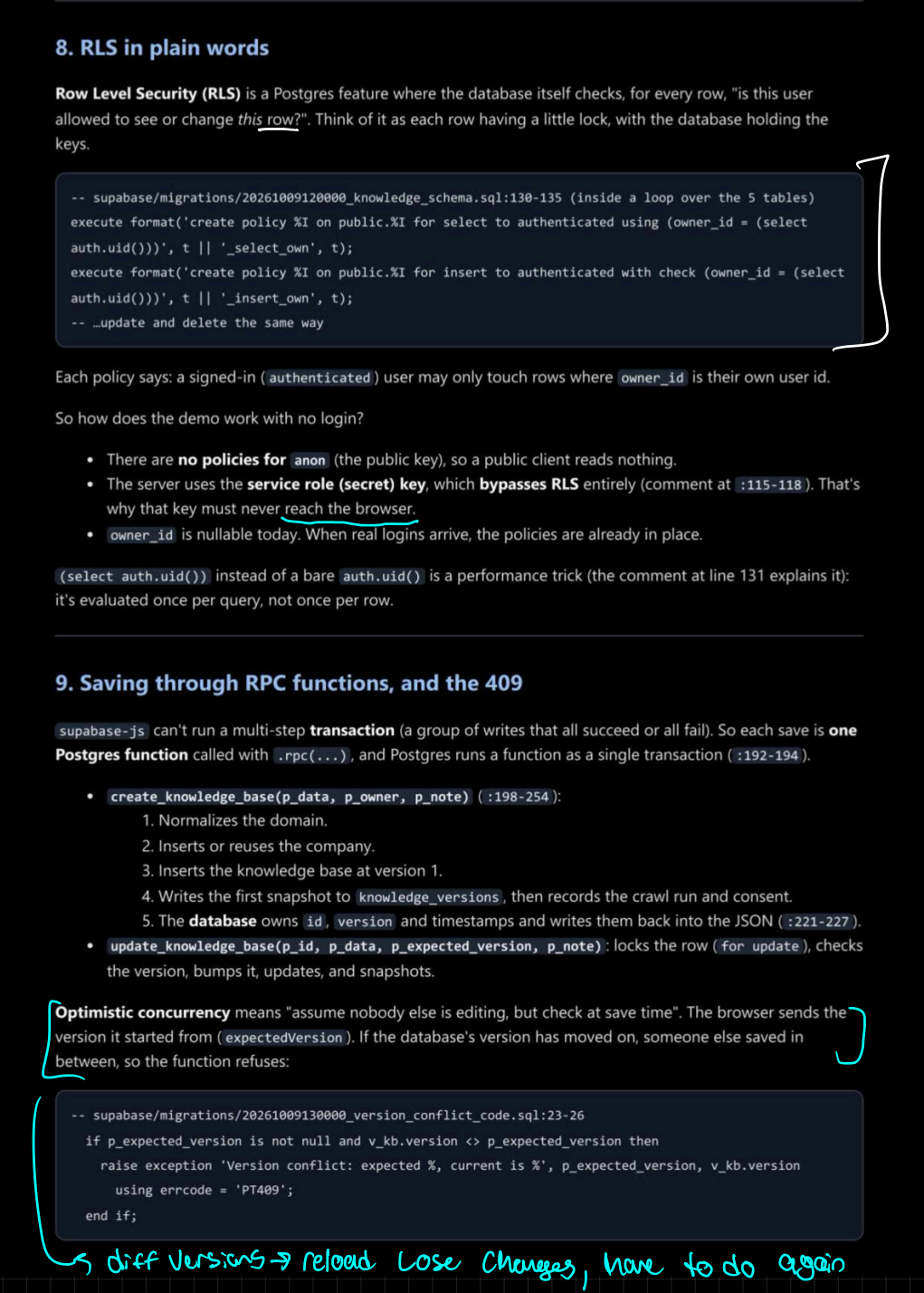
- `ai_enrichments`: an AI answer cache keyed by a hash of prompt + model + input.
- `ai_usage`: one row per UTC day with a counter.

They have no relationships, and nobody but the server can touch them (RLS on, no policies).

> **Why it's built this way (JSONB + copied columns):** the knowledge base has around 60 nested fields that change shape between phases. One JSONB column means no database migration every time a field is added. But searching inside JSON is slow, so the few things the list page filters and sorts on are *copied out* into normal columns, with trigram indexes for `ILIKE '%term%'` search (`:93-96`).

↳ Since its one Jsonb object... with my increased additions I dont have to redesign the table each time i want to add/remove a field(s)

RLS



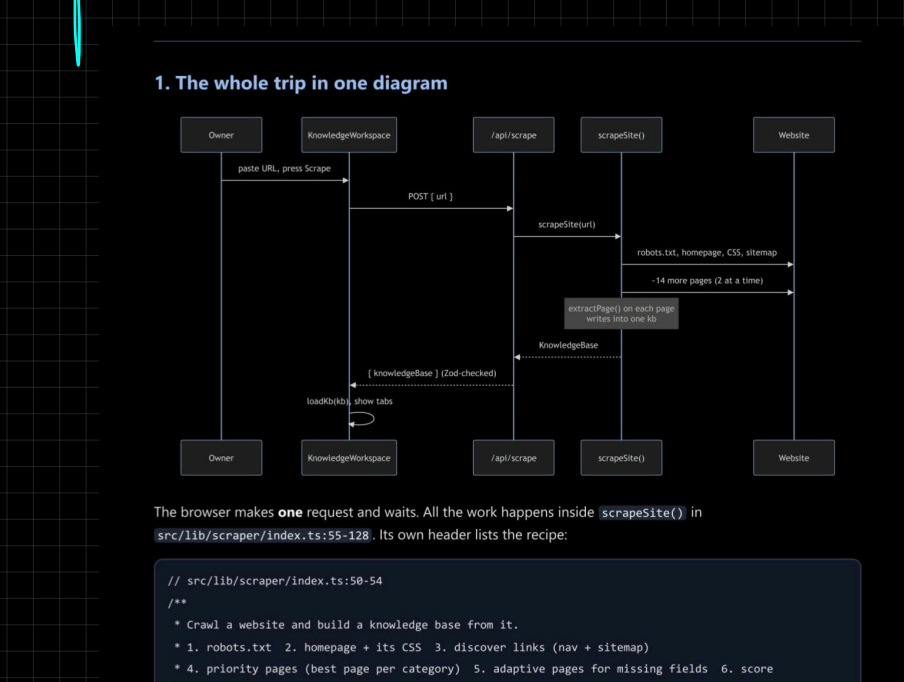
8. RLS in plain words

Row Level Security (RLS) is a Postgres feature where the database itself checks, for every row, "is this user allowed to see or change *this* row?". Think of it as each row having a little lock, with the database holding the keys.

```
-- supabase/migrations/20261009120000_knowledge_schema.sql:130-135 (inside a loop over the 5 tables)
execute format('create policy %I on public.%I for select to authenticated using (owner_id = (select
auth.uid()))', t || '_select_own', t);
execute format('create policy %I on public.%I for insert to authenticated with check (owner_id = (select
auth.uid()))', t || '_insert_own', t);
```

→ diff versions → reload. Lose changes, have to do again

03 The Scrape flow URL → KB

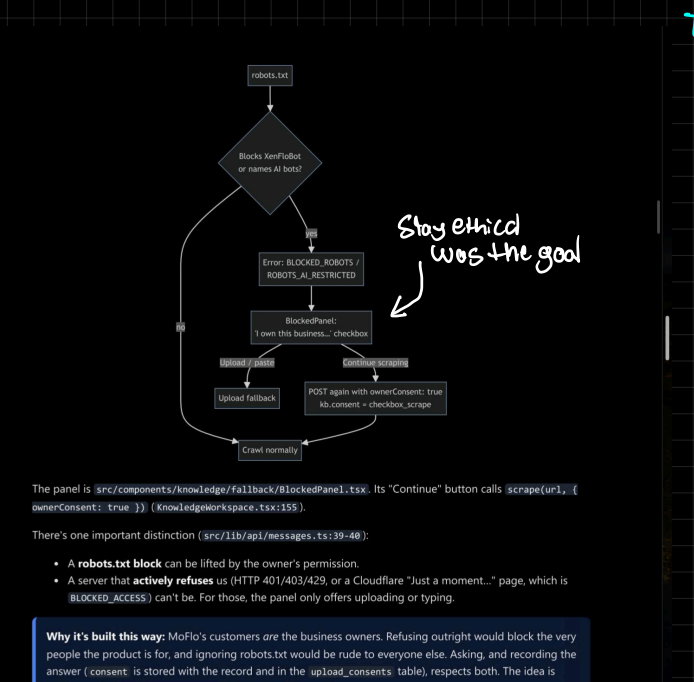


1. The whole trip in one diagram

Owner
KnowledgeWorkspace
/api/scrape
scrapeSite()
Website
paste URL, press Scrape
POST { url }
scrapeSite(url)
robots.txt, homepage, CSS, sitemap
~14 more pages (2 at a time)
extractPage() on each page writes into one kb
KnowledgeBase
{ knowledgeBase } (Zod-checked)
loadKb(kb), show tabs

The browser makes **one** request and waits. All the work happens inside `scrapeSite()` in `src/lib/scraper/index.ts:55-128`. Its own header lists the recipe:

```
// src/lib/scraper/index.ts:50-54
/**
 * Crawl a website and build a knowledge base from it.
 * 1. robots.txt  2. homepage + its CSS  3. discover links (nav + sitemap)
 * 4. priority pages (best page per category)  5. adaptive pages for missing fields  6. score
 */
```



robots.txt
Blocks XenFloBot or names AI bots?
Error: BLOCKED_ROBOTS / ROBOTS_AI_RESTRICTED
BlockedPanel: 'I own this business...' checkbox
Upload / paste
Continue scraping
Upload fallback
POST again with ownerConsent: true kb.consent = checkbox_scrape
Crawl normally

Stay ethical was the goal

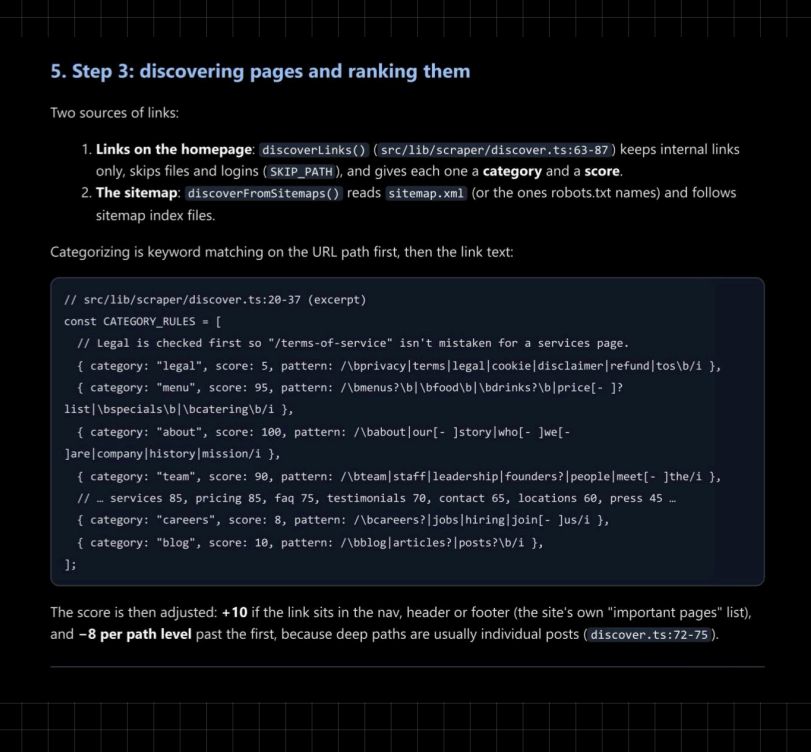
The panel is `src/components/knowledge/fallback/BlockedPanel.tsx`. Its "Continue" button calls `scrape(url, { ownerConsent: true })` (`KnowledgeWorkspace.tsx:155`).

There's one important distinction (`src/lib/api/messages.ts:39-40`):

- A **robots.txt block** can be lifted by the owner's permission.
- A server that **actively refuses** us (HTTP 401/403/429, or a Cloudflare "Just a moment..." page, which is `BLOCKED_ACCESS`) can't be. For those, the panel only offers uploading or typing.

> **Why it's built this way:** MoFlo's customers *are* the business owners. Refusing outright would block the very people the product is for, and ignoring robots.txt would be rude to everyone else. Asking, and recording the answer (`consent` is stored with the record and in the `upload_consents` table), respects both. The idea is written up in `docs/improvements.md` (the Anime Boba row).

Ranking pages



5. Step 3: discovering pages and ranking them

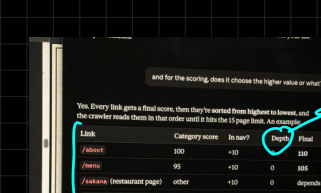
Two sources of links:

1. **Links on the homepage**: `discoverLinks()` (`src/lib/scraper/discover.ts:63-87`) keeps internal links only, skips files and logins (`SKIP_PATH`), and gives each one a **category** and a **score**.
2. **The sitemap**: `discoverFromSitemaps()` reads `sitemap.xml` (or the ones robots.txt names) and follows sitemap index files.

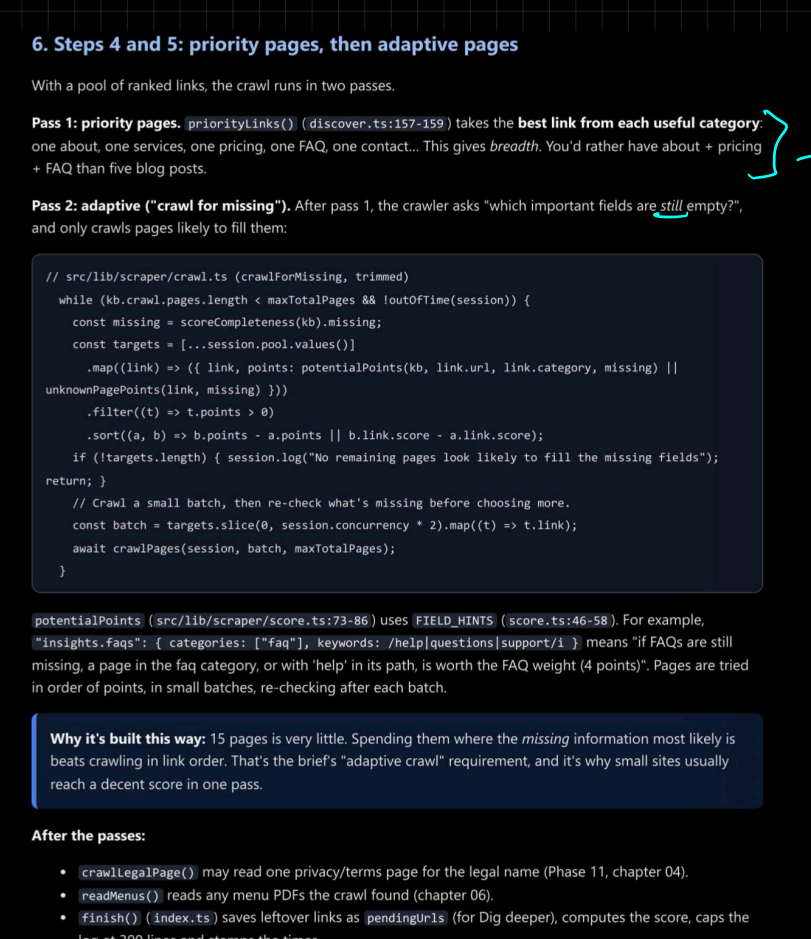
Categorizing is keyword matching on the URL path first, then the link text:

```
// src/lib/scraper/discover.ts:20-37 (excerpt)
const CATEGORY_RULES = [
  // Legal is checked first so "/terms-of-service" isn't mistaken for a services page.
  { category: "legal", score: 5, pattern: /\bprivacy|terms|legal|cookie|disclaimer|refund|tos\b/i },
  { category: "menu", score: 95, pattern: /\bmenus?\b|\bfood\b|\bdrinks?\b|price[- ]?
list|\bspecials\b|\bcatering\b/i },
  { category: "about", score: 100, pattern: /\babout|our[- ]story|who[- ]we[-
]are|company|history|mission/i },
  { category: "team", score: 90, pattern: /\bteam|staff|leadership|founders?|people|meet[- ]the/i },
  // … services 85, pricing 85, faq 75, testimonials 70, contact 65, locations 60, press 45 …
  { category: "careers", score: 8, pattern: /\bcareers?|jobs|hiring|join[- ]us/i },
  { category: "blog", score: 10, pattern: /\bblog|articles?|posts?\b/i },
];
```

The score is then adjusted: **+10** if the link sits in the nav, header or footer (the site's own "important pages" list), and **−8 per path level** past the first, because deep paths are usually individual posts (`discover.ts:72-75`).



how far away from start pg.



6. Steps 4 and 5: priority pages, then adaptive pages

With a pool of ranked links, the crawl runs in two passes.

Pass 1: priority pages. `priorityLinks()` (`discover.ts:157-159`) takes the **best link from each useful category**: one about, one services, one pricing, one FAQ, one contact... This gives *breadth*. You'd rather have about + pricing + FAQ than five blog posts.

→ Sometimes relevant data will be in a weird spot

Pass 2: adaptive ("crawl for missing"). After pass 1, the crawler asks "which important fields are *still* empty?", and only crawls pages likely to fill them:

thats what pass 2 is for

```
// src/lib/scraper/crawl.ts (crawlForMissing, trimmed)
  while (kb.crawl.pages.length < maxTotalPages && !outOfTime(session)) {
    const missing = scoreCompleteness(kb).missing;
    const targets = [...session.pool.values()]
      .map((link) => ({ link, points: potentialPoints(kb, link.url, link.category, missing) ||
unknownPagePoints(link, missing) }))
      .filter((t) => t.points > 0)
      .sort((a, b) => b.points - a.points || b.link.score - a.link.score);
    if (!targets.length) { session.log("No remaining pages look likely to fill the missing fields");
return; }
    // Crawl a small batch, then re-check what's missing before choosing more.
    const batch = targets.slice(0, session.concurrency * 2).map((t) => t.link);
    await crawlPages(session, batch, maxTotalPages);
  }
```

`potentialPoints` (`src/lib/scraper/score.ts:73-86`) uses `FIELD_HINTS` (`score.ts:46-58`). For example, `"insights.faqs": { categories: ["faq"], keywords: /help|questions|support/i }` means "if FAQs are still missing, a page in the faq category, or with 'help' in its path, is worth the FAQ weight (4 points)". Pages are tried in order of points, in small batches, re-checking after each batch.

> **Why it's built this way:** 15 pages is very little. Spending them where the *missing* information most likely is beats crawling in link order. That's the brief's "adaptive crawl" requirement, and it's why small sites usually reach a decent score in one pass.

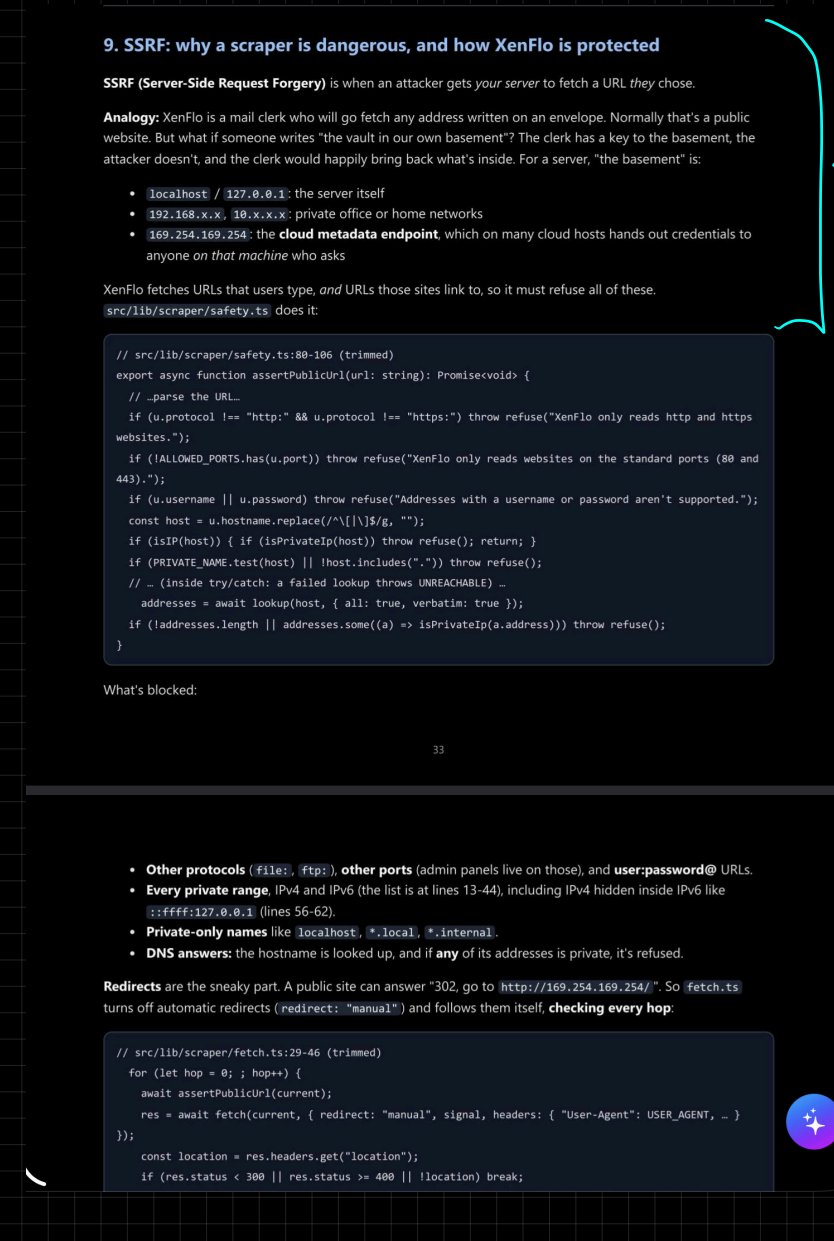
After the passes:

- `crawlLegalPage()` may read one privacy/terms page for the legal name (Phase 11, chapter 04).
- `readMenus()` reads any menu PDFs the crawl found (chapter 06).
- `finish()` (`index.ts`) saves leftover links as `pendingUrls` (for Dig deeper), computes the score, caps the log at 300 lines and stamps the times.

general Idea

- only 15 pages base
- then 15 more up to 200, ~~(useful pages)~~ but only 15 each time

Changed structure



9. SSRF: why a scraper is dangerous, and how XenFlo is protected

SSRF (Server-Side Request Forgery) is when an attacker gets *your server* to fetch a URL *they* chose.

Analogy: XenFlo is a mail clerk who will go fetch any address written on an envelope. Normally that's a public website. But what if someone writes "the vault in our own basement"? The clerk has a key to the basement, the attacker doesn't, and the clerk would happily bring back what's inside. For a server, "the basement" is:

- `localhost` / `127.0.0.1`: the server itself
- `192.168.x.x`, `10.x.x.x`: private office or home networks
- `169.254.169.254`: the **cloud metadata endpoint**, which on many cloud hosts hands out credentials to anyone *on that machine* who asks

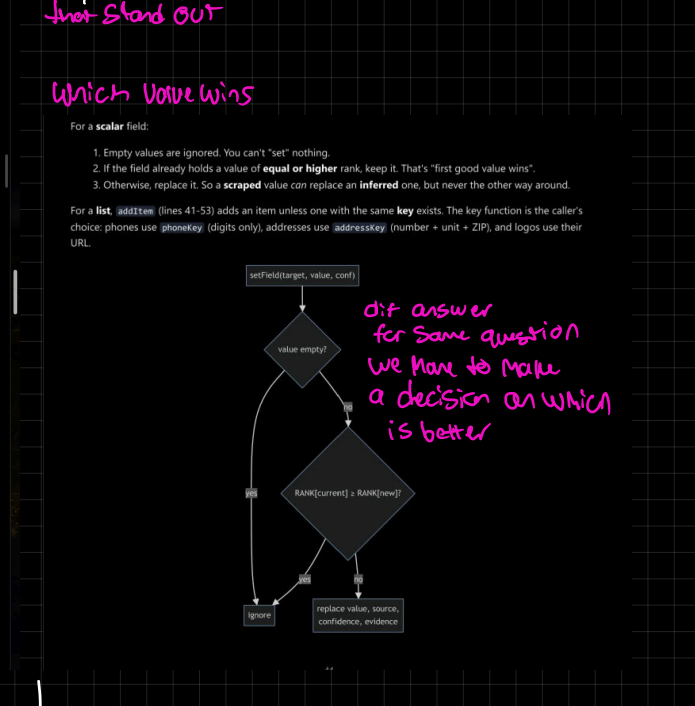
XenFlo fetches URLs that users type, *and* URLs those sites link to, so it must refuse all of these. `src/lib/scraper/safety.ts` does it:

→ talked about this

```
// src/lib/scraper/safety.ts:80-106 (trimmed)
export async function assertPublicUrl(url: string): Promise<void> {
  // …parse the URL…
  if (u.protocol !== "http:" && u.protocol !== "https:") throw refuse("XenFlo only reads http and https
websites.");
  if (!ALLOWED_PORTS.has(u.port)) throw refuse("XenFlo only reads websites on the standard ports (80 and
443).");
  if (u.username || u.password) throw refuse("Addresses with a username or password aren't supported.");
  const host = u.hostname.replace(/^\[|\]$/g, "");
  if (isIP(host)) { if (isPrivateIp(host)) throw refuse(); return; }
  if (PRIVATE_NAME.test(host) || !host.includes(".")) throw refuse();
  // … (inside try/catch: a failed lookup throws UNREACHABLE) …
```

04 The extractors ∴ Not all Just some that stand out

Which value wins



For a **scalar** field:

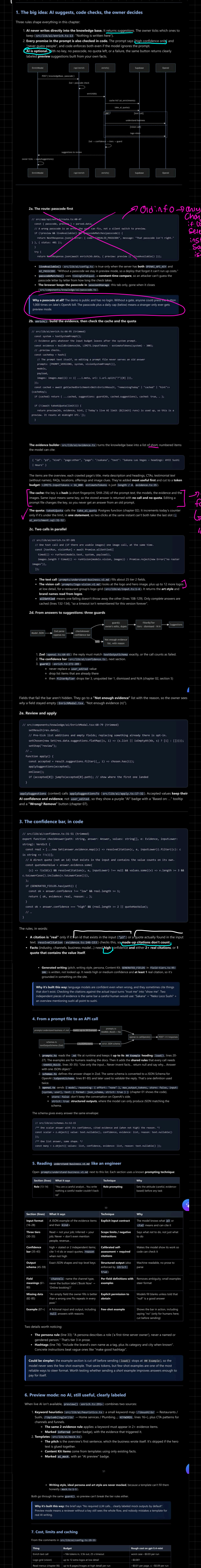
1. Empty values are ignored. You can't "set" nothing.
2. If the field already holds a value of **equal or higher** rank, keep it. That's "first good value wins".
3. Otherwise, replace it. So a **scraped** value *can* replace an **inferred** one, but never the other way around.

For a **list**, `addItem` (lines 41-53) adds an item unless one with the same **key** exists. The key function is the caller's choice: phones use `phoneKey` (digits only), addresses use `addressKey` (number + unit + ZIP), and logos use their URL.

setField(target, value, conf)
value empty?
RANK[current] ≥ RANK[new]?
ignore
replace value, source, confidence, evidence

dif answer for same question we have to make a decision on which is better

05 AI enrichment



1. The big idea: AI suggests, code checks, the owner decides

Three rules shape everything in this chapter:

2a. The route: passcode first

~~`// src/app/api/enrich/route.ts:40-47`~~
```
  const { passcode, preview } = parsed.data;
  // A wrong passcode is an error the user can fix, not a silent switch to preview.
  if (!preview && liveAvailable() && !passcodeMatches(passcode)) {
    return NextResponse.json({ error: { code: "INVALID_PASSCODE", message: "That passcode isn't right."
} }, { status: 401 });
  }
  try {
    return NextResponse.json(await enrich(kb.data, { preview: preview || !liveAvailable() }));
```

→ Old info → only changes if user selects preview instead it'll say that passcode isn't right

- `liveAvailable()` (`src/lib/ai/config.ts`) is true only when the server has **both** `OPENAI_API_KEY` and `AI_PASSCODE`. "Without a passcode we stay in preview mode, so a deploy that forgot it can't run up costs."
- `passcodeMatches()` uses `timingSafeEqual`, a **constant-time compare**, so an attacker can't guess the passcode letter by letter from how long the check takes.
- **The browser keeps the passcode in `sessionStorage`**: this tab only, gone when it closes (`src/components/knowledge/ai/passcode.ts`).

> **Why a passcode at all?** The demo is public and has no login. Without a gate, anyone could press the button 1,000 times on Jake's OpenAI bill. The passcode plus a daily cap (below) means a stranger only ever gets preview mode.

2b. `enrich()`: build the evidence, then check the cache and the quota

```
// src/lib/ai/enrich.ts:66-95 (trimmed)
  const system = textSystemPrompt();
  // Evidence gets whatever the input budget leaves after the system prompt.
  const evidence = buildEvidence(kb, LIMITS.inputTokens - estimateTokens(system) - 300);
  // …preview checks…
  const cacheKey = hash({
    // The prompt text itself, so editing a prompt file never serves an old answer
    prompts: [PROMPT_VERSIONS, system, visionSystemPrompt()],
    models,
    payload,
    images: images.map((i) => ({ ...i.meta, url: i.url.split("?")[0] })),
  });
  const cached = await getCachedEnrichment<Omit<EnrichResult, "remainingToday" | "cached" | "hint">>
(cacheKey);
  if (cached) return { ...cached, suggestions: guard(kb, cached.suggestions), cached: true, … };

  if (!(await takeAiQuota(limit))) {
    return preview(kb, evidence, hint, [`Today's live AI limit (${limit} runs) is used up, so this is a
preview. It resets at midnight UTC.`]);
  }
```

The evidence builder (`src/lib/ai/evidence.ts`) turns the knowledge base into a list of short, numbered items the model can cite:

```
{ "id": "p3", "kind": "page:other", "page": "/sakana", "text": "Sakana Las Vegas — headings: AYCE Sushi
| Hours" }
```

The items are the overview, each crawled page's title, meta description and headings, CTAs, testimonial *text* (without names), FAQs, locations, offerings and image clues. They're added **most useful first** and cut to a **token budget** (`LIMITS.inputTokens = 16_000`; `estimateTokens` is just `length / 4`, `evidence.ts:35`).

The cache: the key is a **hash** (a short fingerprint, SHA-256) of the prompt *text*, the models, the evidence and the images. Same input means same key, so the stored answer is returned with **no call and no quota**. Editing a prompt file changes the key, so you never get an answer from an old prompt.

→ fingerprint for data ↳ stays same even "re enriching"

The quota: `takeAiQuota` calls the `take_ai_quota` Postgres function (chapter 02). It increments today's counter only if it's under the limit, in **one statement**, so two clicks at the same instant can't both take the last slot (`…ai_enrichment.sql:31-52`).

2c. Two calls in parallel

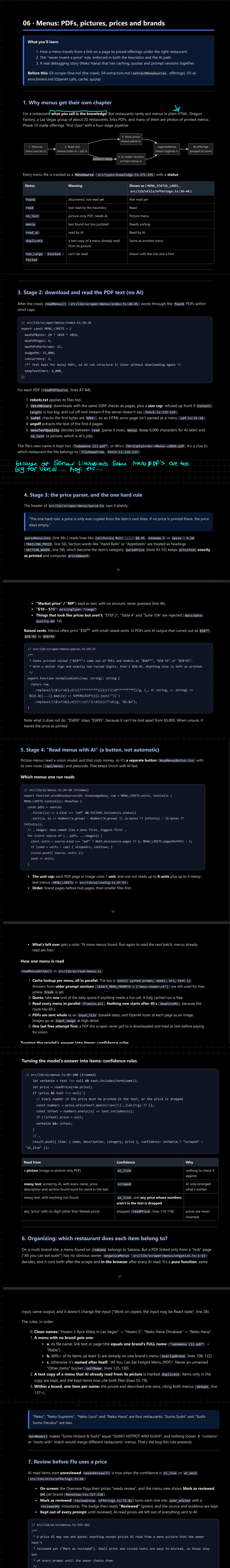
```
// src/lib/ai/enrich.ts:97-101
  // One text call and (if there are usable images) one image call, at the same time.
  const [textRun, visionRun] = await Promise.allSettled([
    timed(() => runText(models.text, system, payload)),
    images.length ? timed(() => runVision(models.vision, images)) : Promise.reject(new Error("no raster
images")),
  ]);
```

- **The text call** (`prompts/understand-business.v1.md`) fills about 25 tier 2 fields.
- **The vision call** (`prompts/logo-vision.v1.md`) looks at the logo and hero image, plus up to 12 more logos at low detail, for a restaurant group's logo grid (`src/lib/ai/input.ts:1-4`). It returns the **art style** and **brand names read from logos**.
- `allSettled` means one failing doesn't throw away the other (lines 108-129). Only *complete* answers are cached (lines 132-134), "so a timeout isn't remembered for this version forever".

2d. From answers to suggestions: three guards

Model JSON
Zod parse (openai.ts)
checkAnswer confidence bar
guard() owner's edits, dupes
filterByTier tiers · dismissed · N/A
Suggestions
fails
'Not enough evidence' list, with reason

06 Menus PDFs, pics, prices/brands



06 · Menus: PDFs, pictures, prices and brands

> **What you'll learn**
>
> 1. How a menu travels from a link on a page to priced offerings under the right restaurant.
> 2. The "never invent a price" rule, enforced in both the heuristics and the AI path.
> 3. A real debugging story (Neko Hana) that ties caching, quotas and prompt versions together.
>
> **Before this:** 03-scrape-flow.md (the crawl), 04-extractors.md (`extractMenuSources`, offerings), 05-ai-enrichment.md (OpenAI calls, cache, quota)

1. Why menus get their own chapter

For a restaurant what you sell *is* the knowledge. But restaurants rarely put menus in plain HTML. Dragon Factory, a Las Vegas group of about 20 restaurants, links PDFs, and many of them are photos of printed menus. Phase 10 made offerings "first class" with a four-stage pipeline:

1. Discover menu-sources.ts
2. Read text menus/index.ts + pdf.ts
3. Parse prices menus/parse.ts
picture / messy
4. AI reader (button) ai/read-menus.ts
organizeMenus menus/organize.ts
kb.offerings grouped by brand

Every menu file is tracked as a `MenuSource` (`src/types/knowledge.ts:271-295`) with a **status**:

```
  /** Text kept for messy PDFs, so AI can structure it later without downloading again */
  keepTextChars: 6_000,
};
```

For each PDF (`readPdfSource`, lines 47-84):

1. **robots.txt** applies to files too.
2. `fetchBinary` downloads with the same SSRF checks as pages, plus a **size cap**: refused up front if `Content-Length` is too big, and cut off mid-stream if the server doesn't say (`fetch.ts:135-139`).
3. `isPdf` checks the first bytes are `%PDF-`, so an HTML error page isn't parsed as a menu (`pdf.ts:15-18`).
4. **unpdf** extracts the text of the first 4 pages.
5. `menuTextQuality` decides between `read` (parse it now), `messy` (keep 6,000 characters for AI later) and `no_text` (a picture, which is AI's job).

The file's own name is kept too: `"nabemenu (2).pdf"`, or Wix's `?dn=Captain+6+-+Menu+-+2026.pdf`. It's a clue to which restaurant the file belongs to (`fileNameFrom`, `fetch.ts:116-133`).

because of server limitations some menu pdf's are too big for vercel ... kogi etc...

4. Stage 3: the price parser, and the one hard rule

The header of `src/lib/scraper/menus/parse.ts` says it plainly:

> "The one hard rule: a price is only ever copied from the item's own lines. If no price is printed there, the price stays empty."

`parseMenuLines` (line 99+) reads lines like `California Roll ..... $8.95`, `Edamame 5` or `Gyoza - 6.50` (`TRAILING_PRICE`, line 56). Section words like "Hand Rolls" or "Appetizers" are treated as headings (`SECTION_WORDS`, line 58), which become the item's category. `parsePrice` (lines 43-53) keeps `priceText` **exactly as printed** and computes `priceAmount`:

55

- **"Market price" / "MP":** kept as text, with no amount, never guessed (line 46).
- **"$10 – $15":** `pricingType: "range"`.
- **Things that look like prices but aren't:** "STEP 2", "Table 4" and "Suite 104" are rejected (`docs/data-quality.md` 1d).

Raised cents. Menus often print "$58⁹⁵" with small raised cents. In PDFs and AI output that comes out as `$58⁹⁵`, `$58 95` or `$58^95`:

```
// src/lib/scraper/menus/parse.ts:29-37
/**
 * Cents printed raised ("$58⁹⁵") come out of PDFs and models as "$58⁹⁵", "$58 95" or "$58^95".
 * With a dollar sign and exactly two raised digits, that's $58.95. Anything else is left as printed.
 */
export function normalizeCents(raw: string): string {
  return raw
    .replace(/(\$\s?\d{1,4})([⁰¹²³⁴⁵⁶⁷⁸⁹]{2})(?![\d⁰¹²³⁴⁵⁶⁷⁸⁹])/g, (_, d: string, c: string) =>
`${d}.${[...c].map((x) => SUPERSCRIPT[x]).join("")}`)
    .replace(/(\$\s?\d{1,4})(?:\s|\^)(\d{2})(?!\d)/g, "$1.$2");
}
```

Note what it does *not* do: "$5895" stays "$5895", because it can't be told apart from $5,895. When unsure, it leaves the price as printed.

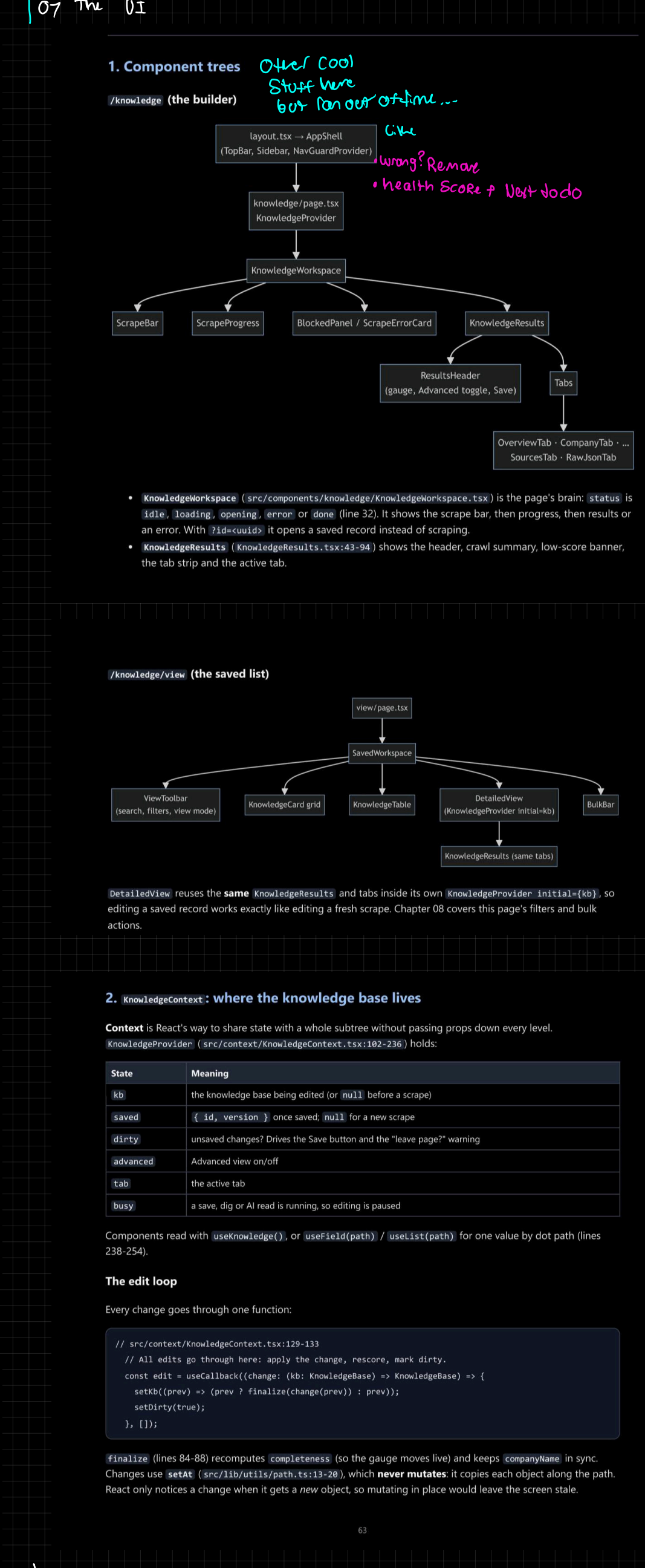
5. Stage 4: "Read menus with AI" (a button, not automatic)

Picture menus need a vision model, and that costs money, so it's **a separate button** (`ReadMenusButton.tsx`) with its own route (`/api/menus`) and passcode. That keeps Enrich with AI fast.

Which menus one run reads

```
// src/lib/ai/menus.ts:34-58 (trimmed)
export function pickMenuSources(kb: KnowledgeBase, cap = MENU_LIMITS.units, textCalls =
MENU_LIMITS.textCalls): MenuPlan {
  const pdfs = sources
    .filter((s) => s.kind === "pdf" && PICTURE.includes(s.status))
```

07 The UI



1. Component trees

Other cool stuff here but ran out of time...
like
- wrong? Remove
- health score + next todo

`/knowledge` (the builder)

layout.tsx → AppShell (TopBar, Sidebar, NavGuardProvider)
knowledge/page.tsx KnowledgeProvider
KnowledgeWorkspace
ScrapeBar
ScrapeProgress
BlockedPanel / ScrapeErrorCard
KnowledgeResults
ResultsHeader (gauge, Advanced toggle, Save)
Tabs
OverviewTab · CompanyTab · ... SourcesTab · RawJsonTab

- `KnowledgeWorkspace` (`src/components/knowledge/KnowledgeWorkspace.tsx`) is the page's brain: `status` is `idle`, `loading`, `opening`, `error` or `done` (line 32). It shows the scrape bar, then progress, then results or

Context is React's way to share state with a whole subtree without passing props down every level. `KnowledgeProvider` (`src/context/KnowledgeContext.tsx:102-236`) holds:

State	Meaning
`kb`	the knowledge base being edited (or `null` before a scrape)
`saved`	`{ id, version }` once saved; `null` for a new scrape
`dirty`	unsaved changes? Drives the Save button and the "leave page?" warning
`advanced`	Advanced view on/off
`tab`	the active tab
`busy`	a save, dig or AI read is running, so editing is paused

Components read with `useKnowledge()`, or `useField(path)` / `useList(path)` for one value by dot path (lines 238-254).

The edit loop

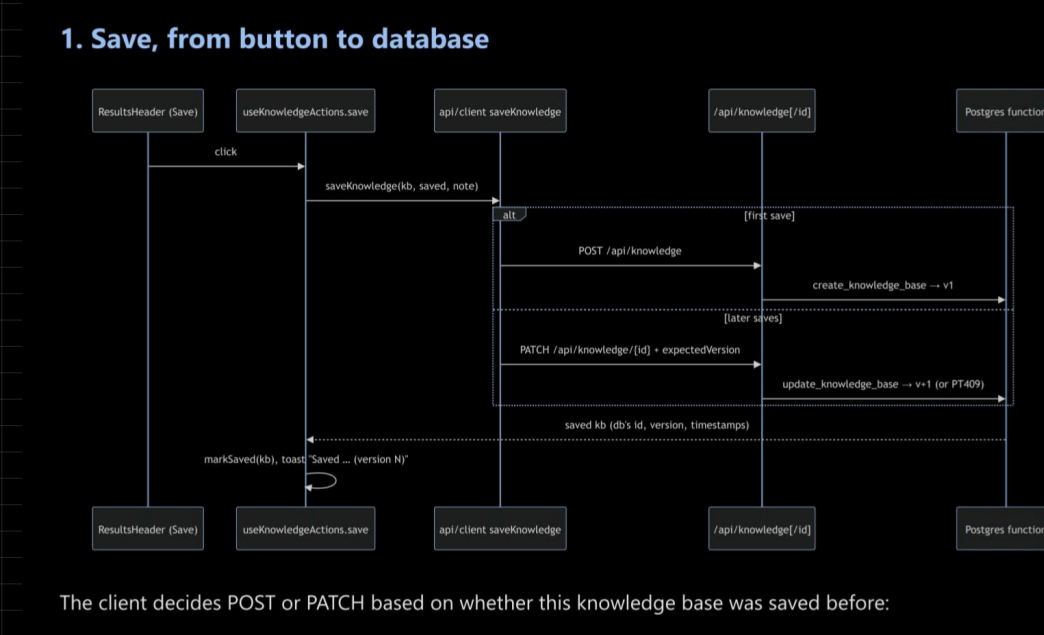
Every change goes through one function:

```
// src/context/KnowledgeContext.tsx:129-133
  // All edits go through here: apply the change, rescore, mark dirty.
  const edit = useCallback((change: (kb: KnowledgeBase) => KnowledgeBase) => {
    setKb((prev) => (prev ? finalize(change(prev)) : prev));
    setDirty(true);
  }, []);
```

`finalize` (lines 84-88) recomputes `completeness` (so the gauge moves live) and keeps `companyName` in sync. Changes use `setAt` (`src/lib/utils/path.ts:13-20`), which **never mutates**: it copies each object along the path. React only notices a change when it gets a *new* object, so mutating in place would leave the screen stale.

63

08 Saving, versions, rescrape



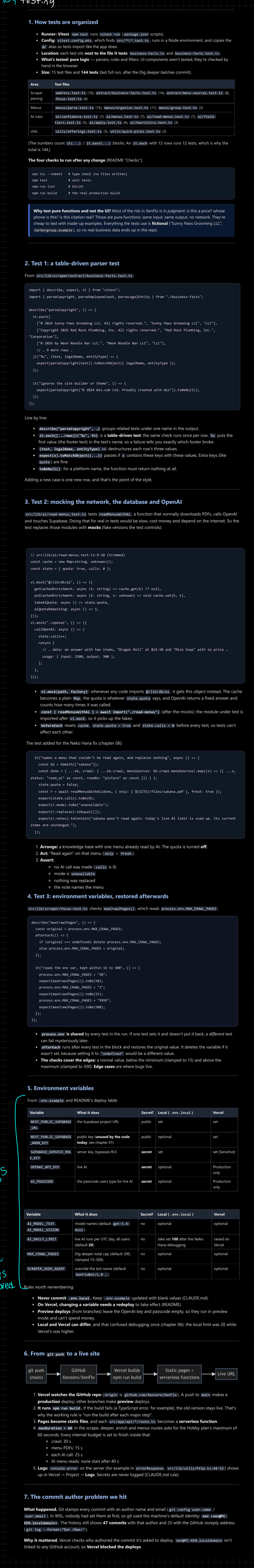
1. Save, from button to database

ResultsHeader (Save)
useKnowledgeActions.save
api/client saveKnowledge
/api/knowledge[/id]
Postgres function
click
saveKnowledge(kb, saved, note)
alt
[first save]
POST /api/knowledge
create_knowledge_base → v1
[later saves]
PATCH /api/knowledge/[id] + expectedVersion
update_knowledge_base → v+1 (or PT409)
saved kb (db's id, version, timestamps)
markSaved(kb), toast "Saved … (version N)"

The client decides POST or PATCH based on whether this knowledge base was saved before:

Version history → Never rewritten

09 testing



1. How tests are organized

- **Runner: Vitest.** `npm test` runs `vitest run` (`package.json` scripts).
- **Config:** `vitest.config.mts`, which finds `src/**/*.test.ts`, runs in a Node environment, and copies the `@/` alias so tests import like the app does.
- **Location:** each test sits **next to the file it tests**: `business-facts.ts` and `business-facts.test.ts`.
- **What's tested: pure logic** — parsers, rules and filters. UI components aren't tested; they're checked by hand in the browser.
- **Size:** 15 test files and **144 tests** (last full run, after the Dig deeper batches commit).

Area	Test files
Scraper parsing	`address.test.ts` (16), `extract/business-facts.test.ts` (14), `extract/menu-sources.test.ts` (8), `focus.test.ts` (6)
Menus	`menus/parse.test.ts` (15), `menus/organize.test.ts` (11), `menus/group.test.ts` (3)
AI rules	`ai/confidence.test.ts` (7), `ai/menus.test.ts` (7), `ai/read-menus.test.ts` (7), `ai/field-tiers.test.ts` (5), `ai/apply.test.ts` (4), `ai/heuristics.test.ts` (3)
Utils	`utils/offerings.test.ts` (3), `utils/quick-picks.test.ts` (3)

(The numbers count `it(...)` / `it.each(...)` blocks. An `it.each` with 12 rows runs 12 tests, which is why the total is 144.)

The four checks to run after any change (README "Checks"):

```
npx tsc --noEmit   # type check (no files written)
npm test           # unit tests
npm run lint       # ESLint
npm run build      # the real production build
```

5. Environment variables

From `.env.example` and README's

My keys etc → Used a local env.file so no keys were shared/pushed

Rules worth remembering:

6. From `git push` to a live site